Shelf Ready PRD v3.2

Shelf Ready — PRD v3.2 (final)

PayPal AI Hackathon 2026 · Sandbox only · Version 3.2 · 24 Oct 2026

Tagline: Shelf Ready is the crash test and repair shop for small online stores that want AI agents to buy from them.

One-line pitch. Same agent, same task: it fails on your store today and succeeds after Shelf Ready fixes it, with proof for every number.

Item	Decision
Hackathon	PayPal AI Hackathon 2026 on Devpost. Five criteria: Technological Implementation, Design, Potential Impact, Innovation/Idea, Presentation.
Hard deadline	12 Nov 2026, 12:00 pm PST (= 13 Nov, 1:00 am PKT). Submit on 11 Nov (PKT). 12 Nov is emergency buffer only.
Demo must stay live	Until the end of judging (15 Dec 2026), free, no restriction.
Environment	PayPal sandbox only. No real money. Stated in README and on the landing page.
Licence / language	MIT, visible in the repo About section. All materials in English.
Repository	github.com/Eman2123/Shelf-Ready (public, MIT)
Backend	**FastAPI (Python 3.11+)**. Frontend: Next.js 14 + TypeScript (UI only).

1. Hackathon fit

1.1 Requirements and how we meet them

Requirement	How we meet it
Meaningful PayPal integration (sandbox)	Catalog Products, Orders v2 (authorize, capture, void, refund), verified Webhooks, PayPal MCP / Agent Toolkit as buyer-agent tools and as an independent read-only witness.
Meaningful AI	AI audit rubric, AI fix extraction with provenance, buyer agent. Deterministic code decides every money action.
Working prototype	Seeded no-login demo, hosted URL, plus setup steps in README (under 10 steps).
Text description	Features, benchmark and mechanism, written to stand alone.
Tools section	README and Devpost name every model and AI service and what each does.
Public repo + licence	GitHub, MIT, licence visible in About.
Demo video	Under 3 minutes, public on YouTube, shows the project running, no copyrighted music, no third-party trademarks.
Free access until judging ends	Hosted demo kept alive; recorded replay mode so live-model limits cannot break it.

1.2 Judging map

Criterion	What we deliver
Technological Implementation	PayPal Catalog + Orders + Webhooks + MCP in one real flow; store MCP server with quote binding; deterministic policy engine; provenance validator; computed readiness score; hash-chained run receipt; tests on money paths.
Design	One coherent flow: Audit, Fix review, Orders + Policy pipeline, Agent test, Report. Shared UI kit, empty and error states.
Potential Impact	Real audience (small sellers worldwide), measurable result (readiness score, completion rate, safe-stop rate) on a published 12-task benchmark.
Innovation / Idea	Agent-readiness measured as a property of the store, not the agent (see section 2.3).
Presentation	One story: fail, fix, succeed, then a $180 order blocked by code.

1.3 Prize strategy

- Rule: one project can win one Grand Prize or one Honorable Mention, plus one Sponsor Prize (re-confirm on the Rules page).
- Page lists more cash prizes than v3.1 tracked. Realistic targets: 1st–3rd, **Best Use of Agentic Commerce**, **Best Use of PayPal + AI**, **Most Impactful**, **Best Demo Delivery**.
- **AG Grid** has five winners (1st $5,000, 2nd $2,000, 3rd $1,000 x3). Use AG Grid for the Audit and Orders grids. AG Studio is P2 only. Decision gate on Day 5: if a spike works, keep a top-tier attempt; if not, keep Community AG Grid as a polished table and aim for a lower tier without spending Tier 1 time on it.
- **Channel3** (shopping/product data sponsor) and **Kernel** are on the sponsor list. Check Resources on Day 1; Kernel stays as Playwright fallback. Render is skipped (card risk).
- AG Studio trial is 45 days. Do not start it early; start it so it covers 15 Dec.

1.4 Difference from PayPal Agent Ready / Store Sync

	PayPal Agent Ready / Store Sync	Shelf Ready
Question	How does my catalog reach AI platforms and how do agent payments get accepted?	Is my product data good enough for an agent to buy correctly, and is that purchase safe for me?
Work	Distribution and payment acceptance	Audit, evidence-only repair, per-agent keys, spend policy, benchmark proof
Relationship	Needs clean data to work well	Makes the data clean before sync, then proves an agent can complete a purchase

Positioning line: *Store Sync gets your catalog in front of agents. Shelf Ready proves the agent can actually buy from it, safely.*

---

2. Problem, audience, innovation

Who: small sellers worldwide (Shopify-style shops, Etsy-style sellers, local stores) using PayPal checkout.

Problem: an agent told to buy a blue medium cotton t-shirt under $30 fails when the page says "from $20", has no stock info, hides shipping and returns, or needs a human at checkout.

Why now: agents have started to buy, protocols (ACP, UCP) are forming, but sellers have no way to know whether they are buyable by an agent.

2.1 Goals

- Measure readiness from real checks and move a messy store from low to high with fixes traceable to evidence or seller input.
- Show the same buyer agent, same tasks, failing before and passing after, from recorded runs.
- Money safety: only deterministic code can create or capture an order.

2.2 Non-goals

Real money, mobile app, support for every platform, new open standards, multi-tenant accounts, signed email approval links.

2.3 Innovation thesis

Agent-readiness is a property of the store, not of the agent. So measure it like any property: hold the agent constant, change the store, record the difference. Shelf Ready is that controlled experiment: it ties every failure to a specific store defect and every fix to specific evidence.

Genuinely novel

1. Readiness measured on the store, not the agent.
2. Counterfactual: same agent, same task, only the store changes.
3. Evidence-bound repair, machine-checked by a validator.
4. One proof chain from source data to PayPal order.

Implementation only (do not oversell): payment safety limits, store MCP, merchant dashboards.

What a normal shopping agent cannot prove: which store defect caused a failure; that changing only the store changed the result; that no invented fact entered the catalog.

---

3. Scope: P0 / P1 / P2 / CUT

If a feature does not strengthen innovation, PayPal depth, readiness, proof, safety or demo impact, it is cut. P1/P2 never damage P0 reliability.

Priority	What
P0	Demo store + CSV import. Computed audit score with check-to-task links. Fix engine + provenance validator + rollback. Store MCP (4 tools) + quote binding + JSON-LD. Catalog sync (minimum visible PayPal Catalog use). Policy engine + agent keys (scopes, limits, expiry, revoke). PayPal authorize / capture / void + verified webhooks + idempotency + read-only witness. Buyer agent (BEFORE / AFTER). 12-task benchmark with k/3 runs. In-app approval queue. Screens: Audit, Fix review, Orders + Policy pipeline, Agent test, Report. Proof chain + `verify_receipt` script.
P1	**External catalog import (public feed or seller-approved) — highest-value P1.** Policy simulator. Keys screen polish. Feed URL import. Defect-variant store set (n=5). Scripted no-LLM attacker. One-page threat model. Rate limiting. Small landing page. Login (GitHub OAuth). CodeQL.
P2	AG Studio dashboard. Compatibility matrix. Webhook log screen. Reconciliation job. 2 ADRs.
CUT	Signed approval links, drift scheduler and alerts, readiness badge, Shopify/Woo importers, refund UI, third agent. (OpenAPI is now automatic from FastAPI, not a task.)

Never cut: provenance, policy engine, agent keys, webhook idempotency, before/after demo, tests on money paths.

---

4. Core mechanisms

4.1 Computed readiness score

Dimension	Weight
Variant, price and currency exact	25
Availability true and consistent across surfaces	15
Shipping and returns machine-readable	15
Agent checkout path (no human-only step)	15
Description accurate and free of injection-like content	10
Identifiers and media	10
Freshness and cross-surface consistency	10

- Every check has: check id, dimension, severity, **linked benchmark task** (e.g. `price_ambiguous` → A2), evidence (field, snippet, URL), pass/fail, points.
- Dimension score = passed weighted checks / total weighted checks. Product score = sum of dimension score x weight.
- **Critical cap:** a Critical failure (price ambiguous, or no purchase path) caps the product score at 60. Store-level gates (no checkout path) cap the store score separately. A pretty but ambiguous store cannot score 91.
- Store score = average of product scores.
- **"Why is this store 91?"** The report shows a waterfall: 100 minus deductions, each with evidence (e.g. "2 products: shipping region NEEDS_SELLER_INPUT").
- **No hard-coded numbers.** Every number in the UI carries a `run_id`, else it shows a SAMPLE badge. A test fails if any rendered number lacks a `run_id`.
- Benchmark correlation: build 5 demo store variants (clean, price ambiguous, stock missing, shipping hidden, injection) and show score vs pass rate. Label it

"n=5 demo variants"; make no strong correlation claim.

4.2 Fix engine + provenance validator

Every fix record: `original`, `proposed`, `source` (URL or field), `evidence_snippet`, `method` (RULE, AI_EXTRACT, SELLER_INPUT), `confidence`, `validation_status`, `created_at`, `version`, `rollback_version`.

- **Provenance validator** is deterministic code: every proposed value must be found in the source text (exact or deterministic normalisation such as inch to cm). Not found → rejected.
- **Confidence comes from the method, not the LLM:** exact match 1.0; deterministic normalisation 0.9; AI_EXTRACT counts only if the validator passes.
- `validation_status`: `PASSED_EXACT`, `PASSED_NORMALIZED`, `REJECTED_NO_SUPPORT`, `NEEDS_SELLER_INPUT`.
- **Visible on the Fix review screen:** three lanes — green (applied, snippet highlighted), red (e.g. AI proposed "100% organic cotton", not in source, blocked by validator), yellow (needs seller input). Permanent header counter: **"Unsupported facts in catalog: 0 (N of N fields verified)"**. The demo must show one red rejection.
- Every apply creates a version; rollback restores the previous version and is logged.

4.3 Quote binding

`create_order` requires a `quote_id` that pins price, stock and a hash of both. If price or stock changed since the quote, the call fails with `PRICE_CHANGED` / `STOCK_CHANGED`. This catches stale price and inventory mismatch structurally.

4.4 Proof chain and Readiness Receipt

- Per-purchase chain = **Purchase Trace**. Whole report = **Readiness Receipt**.
- Chain steps: source snapshot → audit run → fix versions → published agent data → agent decision → quote → policy decision → PayPal order → webhook → final result. Each record has `id`, `inputs`, `output_hash`, `prev_hash`.
- Hashes are used for: SHA-256 of source snapshot and published data (proves the AFTER run used exactly what was published); price hash inside the quote; a hash chain over run events.
- `python -m app.scripts.verify_receipt <report_id>` recomputes the chain and checks every report number against recorded runs.
- **Not used:** blockchain, public-key signatures. The hash chain only detects later edits; a database owner could rewrite it all. Call it **tamper-evident**, never tamper-proof.

4.5 Benchmark (12 tasks)

A. Agent capability (3)

- A1 Exact variant ("cheapest blue tee, M, ≤ $30").
- A2 "From $20" product resolved to exact variant price.
- A3 Ambiguous variant ("blue t-shirt", no size): correct behaviour is to ask, not guess.

B. Merchant safety (9)

- B1 Inventory mismatch (page says in stock, inventory 0).
- B2 Stale price (old quote → `PRICE_CHANGED`).
- B3 Unsupported shipping region.
- B4 Injection in product description.
- B5 Order above limit → `NEEDS_APPROVAL`.
- B6 Daily budget, multi-item basket.
- B7 Quantity above max.
- B8 Restricted category.
- B9 Key revoked mid-run.

Method: record model and version, temperature 0, run each task 3 times, report **k/3**. A single run is never evidence.

Metrics: completion rate (A), safe-stop rate (B), unsafe actions (must be 0 in AFTER), wrong purchases, steps. Both rates are needed because "refuse everything" would game a success-only metric.

BEFORE mode never spends money: the agent records a *purchase intent* and ground truth per task is fixed, so unsafe intents (injection worked) show without a real order.

Scripted attacker (P1): a no-LLM script sends every unsafe action directly to the MCP endpoint, proving safety rests on store code, not on agent judgment.

Credibility: at least one real external catalog is imported and its audit shown as-is, even if mixed. Tasks and ground truth are published in the README.

4.6 Why MCP

MCP is a contract, not the novelty:

1. Typed data (price number + currency, enumerated variants, integer stock with `as_of`) instead of guessing from HTML.
2. Authorization is checked on every call (key, scope, expiry, revoke); HTML scraping has no place for it.
3. Quote binding rides on this route.
4. Any MCP-capable agent works without custom integration.

Measurable demo difference: BEFORE tool = `fetch_page` only; AFTER tools = MCP only. Record tool calls, HTML bytes read, steps. AFTER shows **HTML bytes read: 0**.

4.7 Policy engine and pipeline UX

Rule: *AI recommends. Deterministic code decides money actions.*

- `evaluate_policy()` input is a frozen Pydantic model with numbers and enums only. No free-text field exists. Test: change product text, decision output stays byte-identical.
- Rules: per-order limit, require-approval-above, daily budget, max quantity, allowed categories, agent scopes (read, quote, order), key expiry, kill switch.
- **Pipeline view per request (inside the Orders screen):** Agent request → Identity (key valid, scope, expiry, not revoked) → Data (quote fresh, stock) → Limits (per-order, quantity, category, daily budget) → Decision (ALLOW / NEEDS_APPROVAL / DENY) → PayPal action. Each stage green or red with the number ("$180.00 > $50.00 auto-approve limit, `LIMIT_ORDER`").
- BEFORE mode shows an LLM agent can be swayed by injection; AFTER shows money still does not move.

4.8 Readiness Receipt (report)

Top: readiness before → after (waterfall link). Then agent benchmark (completion, safe-stop, unsafe actions, wrong purchases, k/3). Then issues found / fixed / needing seller input. Then **Unsupported facts introduced = 0 (N fields verified)**. Then policy tests, injection tests, PayPal verification (each passed or failed).

Bottom: "Receipt chain: intact". Every number is clickable and opens its `run_id` and raw record.

---

5. Modules and acceptance (P0 highlights)

Module	Done when
M1 Demo store + import	Messy demo store (fictional brand, 12 products) and CSV import feed the same audit grid. Invalid rows are reported with line numbers. A plain scraping agent fails on the demo store.
M2 Connector	Scan returns 12 normalized products in under 20 s; deduped by SKU and source hash.
M3 Audit	Scores are reproducible and computed from checks. Every check has id, severity, evidence, task link. Unit tests per rule.
M4 Fix engine	All fields in 4.2; validator rejects unsupported values; red rejection demonstrable; versions and rollback logged. Test proves no applied value lacks source or seller input.
M5 Agent-ready layer	Catalog patch, JSON-LD, `/.well-known/agent-store.json`, store MCP (`search_products`, `get_product`, `get_policy`, `create_order`) with quote binding. An MCP client completes a purchase with no HTML parsing.
M6 Policy	Rules in 4.7; decisions logged; injection test passes.
M7 Keys	Shown once, stored hashed. Scopes, limits, expiry. Revoked key fails within one request. Tests for over-budget and over-rate.
M8 Checkout	Intent AUTHORIZE; capture only after ALLOW or seller approval; void on reject or timeout; `PayPal-Request-Id` on every write; webhooks verified through PayPal's verification API, deduped by event ID, tolerant of duplicate / delayed / out-of-order delivery; **read-only MCP witness confirms the order before capture (P0)**. Replay tests prove no double capture.
M9/M10 Agent + benchmark	12 tasks, k/3, BEFORE vs AFTER, stored per run, re-runnable by script, recorded replay.
M12 Dashboard	A first-time visitor can run Scan, Fix, Publish, Test with no instructions.
M13 Approvals	In-app queue only.
M15 Platform	Demo mode, security headers, CI, tests on money paths.

---

6. Screens

Screen	Priority	Notes
Audit grid (AG Grid)	P0	One row per product, score bar, check chips, severity filter, store score in header with waterfall link.
Fix review	P0	Three lanes (green/red/yellow), snippet highlight, unsupported-facts counter, versions, rollback.
Orders + Policy pipeline (AG Grid)	P0	Orders list, approve/reject, PayPal order ID/status, pipeline view per request.
Agent test	P0	Split BEFORE / AFTER, same task, live trace, tool calls, HTML bytes, run-benchmark, replay.
Report (Readiness Receipt)	P0	Section 4.8.
Connect / import	P0	PayPal sandbox app, CSV; empty states.
Keys	P1	Create, scopes, budget, expiry, kill switch.
Policy simulator	P1	"A $42 order would need approval."
Landing	P1	Hero, before/after, how it works, safety, call to action.
Compatibility matrix, webhook log, AG Studio analytics	P2	Only after Day 9 checkpoint.

Seller flow: Connect → Scan → Audit grid → Fix review → Apply and Publish → set Policy → Agent test → Orders → Report.

---

7. Architecture and stack (FastAPI)

Two apps, one repo:

- `backend/` — FastAPI (Python 3.11+), all business logic, PayPal, MCP, agents.
- `frontend/` — Next.js 14 (App Router, TypeScript, Tailwind, shadcn/ui, AG Grid). UI only; no business logic. The messy demo store pages also live here.

Layer	Choice	Note
API framework	FastAPI + Uvicorn, Pydantic v2	Contracts are Pydantic models in `backend/app/contracts`. OpenAPI is generated automatically.
Frontend types	`openapi-typescript` from FastAPI's OpenAPI	One source of truth for contracts.
Database	Postgres (Neon or Supabase free tier), SQLAlchemy 2.0 + Alembic	Keep-alive ping for free-tier pause.
HTTP client	`httpx` (async)	PayPal REST, store pages, feed import.
PayPal	REST: Catalog Products, Orders v2, Webhooks with signature verification	`PayPal-Request-Id` on every write. Tool names and Python availability

	API; OAuth client credentials with auto refresh. PayPal MCP / Agent Toolkit for buyer tools and read-only witness.	verified in the Day 1 spike; fallback is a direct, separately authenticated read-only order lookup labelled as the witness.
AI	`openai` Python SDK against an OpenAI-compatible endpoint (AIML API primary, Gemini backup via provider switch). Pydantic structured outputs, temperature 0, results cached by content hash.	Model must support tool calling and JSON output; verified Day 1. Keys stay server-side.
Store MCP	Official `mcp` Python SDK (FastMCP), streamable HTTP, mounted at `/mcp`	Mounting and lifespan verified in the spike.
Buyer agent	Python tool-calling loop, MCP client session for AFTER, `fetch_page` for BEFORE	Records tool calls, HTML bytes, steps.
Browser automation	Playwright (Python), headless, for sandbox buyer approval and e2e	Runs on the VPS (no serverless limits). Kernel is fallback.
Auth	P0: no-login read-only demo role. P1: GitHub OAuth (Authlib) + session cookie.	
Testing	pytest, pytest-asyncio, `respx` for httpx mocks, recorded PayPal fixtures; Playwright e2e; axe on frontend	Each owner tests their folders.
Quality / security	ruff, mypy (strict on services), pip-audit, CodeQL (Python + JS), structured logs (structlog), security headers, rate limiting (slowapi), CORS locked to the frontend origin	No secrets in logs or repo.
Hosting	**Frontend:** Vercel free tier. **Backend:** Docker Compose on a VPS behind nginx + HTTPS (public URL for webhooks, long-running Playwright, SSE for live traces). Fallback: Render (card risk).	Must stay up until 15 Dec. Replay mode is the default so judges never depend on live model limits.

7.1 Repository structure

```
Shelf-Ready/
  README.md  LICENSE  Makefile  docker-compose.yml  .env.example
  .github/ workflows/ci.yml codeql.yml keepalive.yml  CODEOWNERS  pull_request_template.md
  docs/                PRD, threat model, ADRs
  backend/
    pyproject.toml  alembic.ini
    app/
      main.py                      FastAPI app, routers, /mcp mount, /.well-known
      core/                        config, security headers, logging, rate limit
      contracts/                   Pydantic models agreed Day 1-2
      db/                          models.py, session.py, migrations/
      api/                         routers: stores, audit, fixes, policy, keys,
                                   orders, agent_runs, benchmark, report, webhooks_paypal, health
      services/
        paypal/ policy/ keys/ orders/ approvals/ webhooks/        (Person 1)
        importer/ connector/ audit/ fixes/ agent_ready/ mcp_store/ (Person 2)
        agent/ benchmark/ fixtures/                               (Person 3)
        receipt/                   run events + hash chain (shared: P1 + P3)
        ai/                        provider switch, structured output, cache
    scripts/  seed_demo.py  run_benchmark.py  record_run.py  verify_receipt.py
    tests/
  frontend/
    app/  (landing, dashboard screens, demo-store/)
    components/ui/                 shared UI kit
    lib/api/                       generated types + client
```

Rule: each person edits only their folders. Changes to `backend/app/contracts` need a PR that tags the consumers.

7.2 PayPal API usage

Step	Surface
Seed and sync products	Catalog Products (create, list, show, patch: name, description, type, category, image_url, home_url). Richer data goes through JSON-LD, manifest and store MCP.
Purchase	Orders v2: create (AUTHORIZE) → buyer approval → capture; void on reject/timeout; refund API only.
Order state	Webhooks (approved, capture completed) with signature verification, idempotent handling; order lookup.
Independent check	Read-only MCP lookup as second witness before any capture.
Auth	OAuth client credentials, auto refresh.

8. Safety principles

- AI helps, rules decide. Only deterministic code creates or captures an order.
- AI never invents product facts; missing data becomes `NEEDS_SELLER_INPUT`; the validator enforces it.
- Product text is untrusted. Policy input has no free-text fields; injection tests run in CI.
- Every agent action and policy decision is logged (actor, action, detail, time) and chained.
- Sandbox only, stated in README and landing page.
- Secrets in environment variables only; Pydantic validation on every input; security headers; rate limiting on public endpoints.

9. Data model (summary)

Store, Product (name, description, price, currency, stock, variants, shipping, returns, sku, images, category, source_hash), ScanRun, **ScoreCheck** (check_id, dimension, severity, task_link, evidence, passed, points, run_id), AuditResult, **FixRecord** (fields in 4.2), FixVersion, Policy, Agent, AgentKey (hash, scopes, limits, expiry, revoked_at), AgentUsage, AgentRun (tool_calls, html_bytes, steps), BenchmarkRun and BenchmarkResult (task, k/3, outcome, ground_truth), **Quote** (quote_id, price, stock, hash, expires_at), Order (paypal_order_id, amount, policy_decision, status), ApprovalRequest (status, expires_at), WebhookEvent

(paypal_event_id unique, payload, processed_at), **RunEvent** (id, type, inputs, output_hash, prev_hash), AuditLog, ImportJob, User, Session.

10. Team, contracts, working rules

Three people, 19 days (24 Oct to 11 Nov). Names go on the board on Day 1. **If two people:** drop all P2 and merge duties in this order: PayPal checkout, policy, audit, fixes, store MCP, keys. **If one person:** P0 only, benchmark reduced to 8 tasks (A1–A3, B1, B2, B4, B5, B9), replay mode as default, no scripted attacker.

	Person 1: Money and Platform	Person 2: Data and Agent-ready	Person 3: Agents, UI and Story
Modules	M6, M7, M8, M13, M15	M1 import, M2, M3, M4, M5	M1 demo store, M9, M10, M12 UI, landing
Folders	paypal, policy, keys, orders, approvals, webhooks, CI	importer, connector, audit, fixes, agent_ready, mcp_store	agent, benchmark, fixtures, frontend UI kit, demo store
Also owns	Deploy (VPS, nginx, HTTPS), keep-alive, README money sections, threat model	Data model, seeded demo data, README audit and fix sections	Demo script, video, e2e, accessibility
Tests	Policy, key limits, webhook replay, no double capture, injection	Audit rules, provenance validator, rollback	Full-flow e2e, benchmark determinism, axe

Contracts (Pydantic, agree Day 1-2, never change silently)

Contract	Producer	Consumers
Product, ScoreCheck, FixRecord, FixVersion	P2	P3 (UI, benchmark), P1 (categories)
`evaluate_policy(PolicyInput) -> Decision`	P1	P2 (MCP create_order), P3
`create_guarded_order(key, quote_id, items)`	P1	P2 (store MCP)
`verify_key(key) -> agent_id + limits`	P1	P2, P3
Quote, RunEvent (hash chain)	P1 + P3	everyone
Store MCP tools	P2	P3 (buyer agent), P1 (key checks)
Seeded demo data + benchmark task list	P3	everyone

By end of Day 2 each person publishes stubs for every contract they produce.

Working rules: trunk-based, short branches `feature/<module>`, one review per PR, CODEOWNERS by folder; 15-minute daily sync (finished, blocked, contract changes); every day ends with something that runs end to end; `.env.example` lists every variable; PR template: tests added, contract changed, screenshots for UI. Make targets: `dev`, `test`, `lint`, `migrate`, `seed-demo`, `bench-run`, `bench-record`, `verify-receipt`.

11. Day plan (24 Oct to 11 Nov, Pakistan time)

P0 first. P2 only after the Day 9 checkpoint. Checkpoints (merge to main, run full flow on seeded data): Days 5, 9, 12, 15.

Day	Person 1: Money and Platform	Person 2: Data and Agent-ready	Person 3: Agents, UI and Story
1 Sat 24	Scaffold FastAPI + Docker; PayPal sandbox app, token manager, env	DB + Alembic; LLM tool-calling + JSON test; port contracts to Pydantic	Next.js shell, UI kit started; check Resources (Channel3, Kernel)
2 Sun 25	Orders authorize/capture/void; webhook receiver on public URL; Playwright approval spike on VPS	Catalog create/list/patch; contract stubs (Product, Fix)	UI kit + AG Grid shell; messy demo store
3 Mon 26	Demo-mode role; key model + hashing	CSV importer	Demo store pages; agent harness; **benchmark list frozen (12 tasks)**
4 Tue 27	Policy engine v1 + tests (no free-text input); CI green	Connector + scan; audit checks + tests	Audit grid screen; buyer agent BEFORE mode (fetch_page, HTML bytes)
5 Wed 28	**Checkpoint.**	**Checkpoint.**	**Checkpoint.** AG Grid/Studio go/no-go
6 Thu 29	Idempotency, void paths	Fix engine + provenance validator	Buyer agent AFTER mode (MCP client)
7 Fri 30	Webhook verification; read-only witness	Versions + rollback; red-rejection fixture	Agent test screen (split view)
8 Sat 31	Keys + quote contract with P2; kill switch	Agent-ready layer: Catalog patch, JSON-LD, manifest	Fix review UI (3 lanes + counter)
9 Sun 1	**Checkpoint.** Tier 1 end to end; P2 go/no-go	**Checkpoint.**	**Checkpoint.**
10 Mon 2	Orders screen + policy pipeline view; hash-chain RunEvents	Store MCP complete + quote binding; scan speed	Benchmark runner (k/3), replay mode
11 Tue 3	Rate limiting, security headers; `verify_receipt`	Score waterfall + evidence API	Report (Readiness Receipt); run_id enforcement test
12 Wed 4	**Checkpoint.**	**Checkpoint.**	**Checkpoint.**
13 Thu 5	Webhook replay tests; threat model	External catalog import (P1); README audit/fix sections	End-to-end tests; defect-variant stores

14 Fri 6	Failure cases + injection tests; scripted attacker	Cleanup; P1/P2 item if on track	Accessibility; record real benchmark runs
15 Sat 7	**Feature freeze (morning).** Security pass. Checkpoint.	Data cleanup. Checkpoint.	Polish; empty/error states; checkpoint
16 Sun 8	Deploy; keep-alive; clean-machine README run	README review on clean machine	Demo script final
17 Mon 9	README money sections; Devpost text; Tools section	Review all docs; English check	Record demo video
18 Tue 10	Bug fixes only; check hosted demo and credentials	Bug fixes only; check setup steps	Edit video; upload to YouTube
19 Wed 11	Submit on Devpost (Representative)	Final check	Final check; demo stays alive until 15 Dec

12. Risks and mitigations

Risk	Plan
Scope too big for 19 days	P0/P1/P2/CUT, checkpoints, feature freeze 7 Nov, P2 only after Day 9.
Demo must stay alive to 15 Dec	VPS + Docker restart policy, DB keep-alive, replay as default, document everything in README.
LLM credits or rate limits when judges run live	Replay default, cached results, per-IP rate limit, provider switch (AIML / Gemini), results stored in DB.
Playwright buyer approval flaky	Test on the VPS in the Day 2 spike; Kernel fallback; recorded replay last resort.
FastAPI MCP mount or Python PayPal toolkit gap	Day 1-2 spike; fallback: standalone MCP process behind nginx; direct read-only PayPal lookup as witness.
Benchmark looks staged	Import one real external catalog (P1), publish tasks and ground truth, run k/3, show mixed results honestly.
Overlap with Agent Ready / Store Sync	State the difference in README, Devpost text, video.
Another team has a similar idea (20,000+ participants)	Check project gallery and Discussions weekly; lean on counterfactual + provenance + receipt.
Catalog API has few fields	Rich data lives in feed, JSON-LD and store MCP. Say so plainly.
Third-party trademarks in video	Fictional store brand, short PayPal sandbox screen, no other logos in tabs.
Webhooks cannot reach local dev	Public VPS URL or tunnel; replay script with recorded events.
AG Grid prize distracts from P0	AG Studio is P2; Day 5 gate.

13. Demo

13.1 Wow moment (25 s)

1. **0–8 s:** split screen, same task. Left (BEFORE) red: guesses "from $20", no variant, **FAIL**.
2. **8–16 s:** press Fix. 12 fields with evidence, 1 red rejection, 3 yellow, seller fills 1, Publish.
3. **16–20 s:** right (AFTER) green: 4 tool calls, HTML bytes 0, order $24, PayPal authorized, webhook verified.
4. **20–26 s:** give the agent a $180 order. Pipeline stops at limit: **NEEDS_APPROVAL**, PayPal action: none.

Injection and kill switch go in the benchmark scene for 2 seconds, not in the wow moment.

13.2 Video script (under 3:00)

Time	Scene
0:00–0:25	Wow moment.
0:25–1:05	Problem and scan; audit grid and score waterfall.
1:05–1:55	Evidence and validator rejection; seller input; Apply and Publish.
1:55–2:35	Benchmark (completion, safe-stop, unsafe actions = 0, k/3) and injection test.
2:35–3:00	Readiness Receipt, PayPal verification, `verify_receipt`; impact and open source.

Rules: public on YouTube, under 3 minutes, shows the project running, no copyrighted music, no third-party logos, English. Every number shown is a real recorded result.

14. Positioning

Shelf Ready is the crash test and repair shop for small online stores that want AI agents to buy from them.

1. **One-line:** Same agent, same task: it fails on your store today and succeeds after Shelf Ready fixes it, with proof for every number.
2. **30-second pitch:** AI agents skip small stores because product data is vague and checkout needs a human. Shelf Ready runs the same agent before and after: it diagnoses exactly why the agent fails, repairs data only from evidence, opens a policy-guarded PayPal purchase path, and produces a receipt proving every step. Unsafe orders are stopped by code, not by trust.
3. **3-minute story:** section 13.2.
4. **Innovation thesis:** section 2.3.

5. **Why PayPal:** `AUTHORIZE` is a hold-then-decide pattern that fits agent purchases; webhooks and order lookup give third-party confirmation; sandbox is free.
6. **Why now:** agents have started to buy and protocols (ACP, UCP) are forming, but sellers cannot tell whether they are buyable by an agent.
7. **Versus Agent Ready / Store Sync:** they accept payments and deliver catalogs. We check the data is buyable and prove buying is safe.
8. **Why a judge remembers it:** one agent, one task: fail, fix, success, and a $180 order the code refused.

---

15. Submission checklist

- [ ] Public GitHub repo, MIT licence file visible in About.
- [ ] README that stands alone: setup in under 10 steps, sandbox-only statement, benchmark tasks and ground truth, scoring formula, results, test instructions, `verify_receipt` usage.
- [ ] Hosted URL with no-login seeded demo, alive until 15 Dec 2026; recorded replay available.
- [ ] Text description of features and functionality.
- [ ] Tools section: every model and AI service used (AIML API model via OpenAI-compatible SDK, PayPal MCP / Agent Toolkit, Playwright, any sponsor tool) and exactly how.
- [ ] Public YouTube demo under 3 minutes, link on the form.
- [ ] All teammates added; one eligible Representative submits; prize split agreed in advance.
- [ ] English everywhere; no secrets in repo; CI green; CodeQL and pip-audit clean or documented.
- [ ] Submit on 11 Nov (PKT). Hard deadline 12 Nov 12:00 pm PST (13 Nov 1:00 am PKT).

Prize payout notes: paid to the Representative after required forms; non-US winners may need a W-8BEN; winners cover fees and taxes.

Appendix: changes from v3.1

- Backend moved from Next.js API routes (Node/TS) to **FastAPI**; Zod contracts replaced by Pydantic; OpenAPI generated automatically; Playwright and webhooks run on a VPS, removing the serverless risk.
- Tiers replaced by P0/P1/P2/CUT; signed approval links, drift, badge, importers cut.
- Hard-coded 30 → 90 removed; computed score with evidence, critical cap, waterfall, `run_id` rule.
- Fix engine: full FixRecord, deterministic provenance validator, visible counter and red rejection.
- Added quote binding, proof chain (Purchase Trace, Readiness Receipt, hash chain, `verify_receipt`).
- Benchmark: 12 tasks (A capability, B safety), k/3, BEFORE intents only, scripted attacker (P1).
- MCP justified by measurable difference (HTML bytes read: 0).
- Policy pipeline view inside Orders; injection test is structural.
- Prize strategy updated to the full Devpost prize list; AG Grid multi-tier; Channel3 and Kernel noted.
- Fixed v3.1 inconsistency (landing page owner) and added one-person and two-person fallbacks.